

교과목 2 : 데이터 분석과 머신러닝 / 딥러닝

## 단원 1 : 데이터 분석

DAY2. 데이터 전처리

## 목 차

|                          |    |
|--------------------------|----|
| 1. Numpy 행렬과 벡터 활용 ..... | 3  |
| 2. NumPy 활용 .....        | 22 |

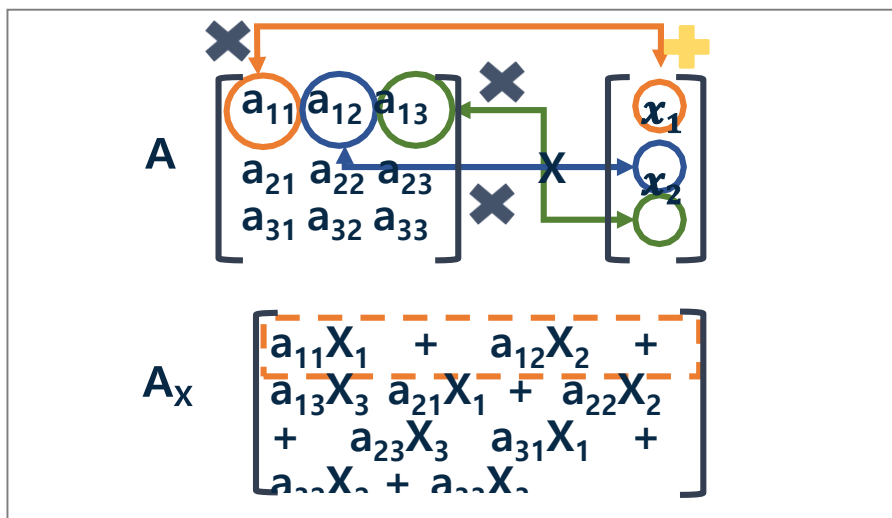
## 1. Numpy 행렬과 벡터 활용

### 1. 행렬과 벡터 간 곱셈하기

#### 1) 행렬과 벡터 간의 곱셈

(1) 행렬 간 곱셈 방식과 동일

(2) 행렬에 있는 원소와 벡터의 성분을 곱하여 계산



[Python 코드]

```
import numpy as np
A=np.array([[1,2,3],[4,5,6],[7,8,9]])
x=np.array([10,20,30])
np.dot(A,x)
```

```
array([140, 320, 500])
```

## (3) 행렬을 열벡터 또는 행벡터로 바꾸어 계산

$$A = \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix} \rightarrow \begin{bmatrix} a_{11} \\ a_{21} \\ a_{31} \end{bmatrix} \quad \begin{bmatrix} a_{12} \\ a_{22} \\ a_{32} \end{bmatrix} \quad \begin{bmatrix} a_{13} \\ a_{23} \\ a_{33} \end{bmatrix}$$
$$A = \begin{bmatrix} v_1 & v_2 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} \rightarrow x_1 v_1 + x_2 v_2 +$$

### [Python 코드]

```
import numpy as np
A=np.array([[1,2,3],[4,5,6],[7,8,9]])
x=np.array([10,20,30])
# 행렬을 열벡터로 변환
v1=A[:,0]
v2=A[:,1]
v3=A[:,2]
np.dot(x[0], v1)+np.dot(x[1], v2)+np.dot(x[2], v3)

array([140, 320, 500])
```

## 2) 크기가 다른 행렬의 곱셈

- (1)  $m \times n$ 인 행렬과  $n \times p$ 인 행렬을 곱하면  $m \times p$  행렬이 만들어짐
- (2) 한 행렬의 열크기(column)와 곱할 행렬의 행크기(row)가 서로 같아야 함
- (3) dot 함수를 활용하여 구할 수 있음(\*은 Error 발생)

```
import numpy as np
A=np.array([[1,2,3],[4,5,6]]) #2행 3열
B=np.array([[1,2],[3,4],[5,6]]) #3행 2열
#2행2열
np.dot(A,B)

array([[22, 28],
       [49, 64]])
```

### ▶ 여기서 잠깐

- (1) 행렬의 곱셈 연산자 \* → 크기가 서로 다른 행렬 곱(Error!)

```
import numpy as np
A=np.array([[1,2,3],[4,5,6]]) #2행 3열
B=np.array([[1,2],[3,4],[5,6]]) #3행 2열
#2행2열
A*B
```

```
-----
ValueError                                Traceback (most recent call last)
<ipython-input-10-85ca24dd4820> in <module>()
      3 B=np.array([[1,2],[3,4],[5,6]]) #3행 2열
      4 #2행2열
----> 5 A*B

ValueError: operands could not be broadcast together with shapes (2,3) (3,2)
```

## 3) 스칼라와 벡터의 곱셈

(1) 스칼라(상수)를 벡터의 성분과 곱하여 계산

$$V \begin{bmatrix} v \\ 1 \\ v \\ 2 \\ \vdots \end{bmatrix} \quad a \quad = \quad aV \begin{bmatrix} a \times v_1 \\ a \times v_2 \\ a \times v_3 \end{bmatrix}$$

```
import numpy as np
V=np.array([[1],[2],[3]]) #3행 1열(열벡터)
a=10
a*V #np.dot(a,V)
```

array([[10],  
 [20],  
 [30]])

## 2. 선형 연립방정식 풀기

### 1) 선형 연립방정식이란?

- (1) 둘 이상의 선형 방정식을 묶어서 선형 연립방정식을 만듦
- (2) 선형방정식: 최고차 항의 차수가 1을 넘지 않는 다항방정식
- (3) 변수가 n개인 선형방정식

$$a_1 x_1 +$$



선형방정식을 행렬로 표현

열벡터를 기본으로 하므로 a는 전치된 벡터로 나타냄

$$[a_1 \ a_2 \ a_3 \ \dots \ a_n] \begin{bmatrix} x_1 \\ x_2 \\ \vdots \end{bmatrix} = a^T x =$$

## (4) 다음과 같은 연립방정식을 행렬로 표현하기

$$\begin{array}{l}
 a_{11} x_1 + \\
 a_{12}x_2 + a_{13}x_3 + \dots + a_{1n}x_n = b_1 \\
 \\
 a_{21} x_1 + \\
 a_{22}x_2 + a_{23}x_3 + \dots + a_{2n}x_n = b_2
 \end{array}$$

↓

선형방정식을 행렬로 표현

$$\begin{bmatrix}
 a_{11} & a_{12} & a_{13} & \dots & a_{1n} \\
 a_{21} & a_{22} & a_{23} & \dots & a_{2n} \\
 \dots & & & & 
 \end{bmatrix}
 \begin{bmatrix}
 x_1 \\
 x_2 \\
 x_3 \\
 \dots
 \end{bmatrix}
 =
 \begin{bmatrix}
 b_1 \\
 b_2 \\
 b_3 \\
 \dots \\
 b_n
 \end{bmatrix}
 =$$

## (5) 연립방정식 풀기

$$\begin{array}{l}
 a_{11} x_1 + \\
 a_{12}x_2 + a_{13}x_3 + \dots + a_{1n}x_n = b_1 \\
 a_{21} x_1 + a_{22}x_2 + a_{23}x_3 + \dots + a_{2n}x_n = b_2 \\
 \dots
 \end{array}$$



## ① 선형방정식을 행렬로 표현

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} \dots a_{1n} \\ a_{21} & a_{22} & a_{23} \dots a_{2n} \\ \dots & \dots & \dots \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ \dots \end{bmatrix} = \begin{bmatrix} b_1 \\ b_2 \\ b_3 \\ \dots \\ b_m \end{bmatrix} =$$

## ② 역행렬을 양변에 곱하여 x(해)를 구함

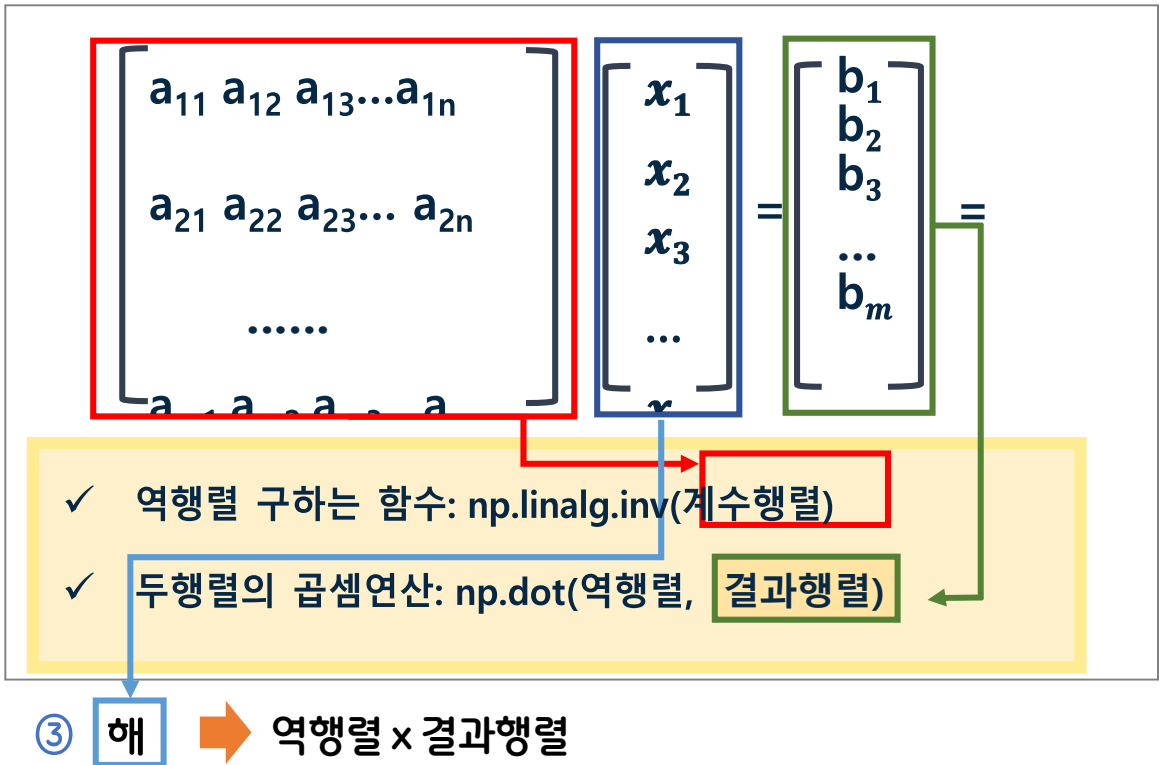
- $Ax = b$  의 양변에 역행렬을 곱함

$$A^{-1}Ax = A^{-1}b$$

$$x = A^{-1}b$$

## (6) 연립방정식 해 풀기

- ① 주어진 연립방정식을 행렬로 표현
- ② 역행렬을 구하여 양변에 곱함



## 2) 선형방정식의 개수와 미지수의 개수가 같은 경우

(1) inv와 dot를 이용하여 풀기

- $4x_1 + 3x_2 = 23$
- $3x_1 + 2x_2 = 16$

```
import numpy as np
A = np.array([[4, 3], [3, 2]])
b = np.array([23, 16])
invA=np.linalg.inv(A)
x=np.dot(invA,b)
x
```

array([2., 5.])

[해가 올바른지 확인: allclose]

`np.allclose(np.dot(A, x), b)` → 올바른 경우, True

## 2) 선형방정식의 개수와 미지수의 개수가 같은 경우

(2) solve를 이용하여 풀기

[형식]

`numpy.linalg.solve(계수행렬, 결과행렬)`

- $4x_1 + 3x_2 = 23$

- $3x_1 + 2x_2 = 16$

```
import numpy as np
A = np.array([[4, 3], [3, 2]])
b = np.array([23, 16])
x=np.linalg.solve(A,b)
x
```

```
array([2., 5.])
```

## 3) 선형방정식의 개수와 미지수의 개수가 다른 경우

(1) `np.linalg.lstsq`를 이용하여 해를 구함

[형식]

`numpy.linalg.lstsq(계수행렬, 결과행렬, rcond=None)[0]`

- $1x_1 + 4x_2 + 3x_3 = 23$
- $1x_1 + 3x_2 + 2x_3 = 16$



```
import numpy as np
A = np.array([[1, 4, 3], [1, 3, 2]])
b = np.array([23, 16])
x=np.linalg.lstsq(A,b, rcond=None)[0]
x
```

```
array([-1.,  3.,  4.])
```

## 3.행렬식 계산하기

### 1) 행렬식이란?

#### (1) 행렬식의 정의

- 정사각 행렬에 스칼라를 대응시키는 함수
- 행렬 A의 행렬식:  $|A|$  또는  $\det(A)$ 로 표현
- $1 \times 1$ 인 행렬의 행렬식은 자기 자신임
- $2 \times 2$ 인 행렬의 행렬식

$$\det \begin{bmatrix} a & b \\ c & d \end{bmatrix} = ad - bc$$

#### (2) 행렬식 만들기

- $2 \times 2$  행렬의 행렬식 만들기

$$\det \begin{bmatrix} a & b \\ c & d \end{bmatrix} = ad - bc$$

- $3 \times 3$  행렬의 행렬식 만들기

$$\det \begin{bmatrix} a & b & c \\ d & e & f \\ g & h & i \end{bmatrix}$$

## (2) 행렬식 만들기

### ① 3x3 행렬을 2x2 행렬로 변형

- 제(1,1) 소행렬식을 구함

$$\det \begin{bmatrix} a & b & c \\ d & \boxed{e} & f \\ g & h & i \end{bmatrix} \rightarrow \det \begin{bmatrix} e & f \\ h & i \end{bmatrix} = ei - fh$$

- 제(1,2) 소행렬식을 구함

$$\det \begin{bmatrix} a & b & c \\ \boxed{d} & e & \boxed{f} \\ g & h & i \end{bmatrix} \rightarrow \det \begin{bmatrix} d & f \\ g & i \end{bmatrix} = di - fg$$

- 제(1,3) 소행렬식을 구함

$$\det \begin{bmatrix} a & b & c \\ d & e & f \\ \boxed{g} & \boxed{h} & i \end{bmatrix} \rightarrow \det \begin{bmatrix} d & e \\ g & h \end{bmatrix} = dh - eg$$

## ② 지금까지 구한 소행렬식에 0행의 요소를 곱하여 구함

$$\begin{aligned} & a \times \text{제}(1,1) \text{ 소행렬식} - b \times \text{제}(1,2) \text{ 소행렬식} + c \times \text{제}(1,3) \text{ 소행렬식} \\ &= a(ei-fh) - b(di-fg) + c(dh-eg) \\ &= aei - afh - bdi + bfg + cdh - ceg \end{aligned}$$

### ▶ 여기서 잠깐

(1) 소행렬이란? → 지정 행과 열을 제거하여 구성하는 정방행

- 예 : A의 (i, j) 소행렬
- A에서 i번째 행과 j번째 열을 제거하여 구성되는 (n-1)차 정방행렬



## 2) 행렬식 구하기

### (1) det 함수 사용

[형식]

`numpy.linalg.det(행렬)`

$$\begin{bmatrix} 1 & 4 \\ 1 & 3 \end{bmatrix} \rightarrow \det \begin{bmatrix} 1 & 4 \\ 1 & 3 \end{bmatrix} \rightarrow (1 \times 3) - (1 \times 4) = -1$$

```
import numpy as np  
A = np.array([[1, 4], [1, 3]])  
np.linalg.det(A)
```

-1.0

## 2) 행렬식 구하기

▶ 여기서 잠깐

(1)  $\det$  함수는 어떤 때 활용하는가?

→ 연립방정식의 해나 역행렬을 구할 때 사용

- 연립방정식의 계수행렬에 대한 행렬식이 0이 아니면  
연립방정식은 해를 가짐
- 행렬식이 0이면, 연립방정식은 무수히 많은 해를  
가지거나 해가없음
- 행렬식이 0일 아닐 때, 역행렬이 존재

# 데이터 전처리

## 3) 3차 정사각행렬 행렬식 구하기

$$\begin{bmatrix} 8 & 5 & 3 \\ 4 & 1 & 6 \\ 7 & 10 & 9 \end{bmatrix} \quad \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix}$$

(1) 3x3 행렬을 2x2 행렬로 변형

① 제(1,1), 제(1,2), 제(1,3) 소행렬식을 구함

$$\begin{bmatrix} 8 & 5 & 3 \\ 4 & 1 & 6 \\ 7 & 10 & 9 \end{bmatrix} \rightarrow \det \begin{bmatrix} 1 & 6 \\ 10 & 9 \end{bmatrix} \rightarrow (1 \times 9) - (6 \times 10) = 9 - 60 = -51$$

## 3) 3차 정사각행렬 행렬식 구하기

(2)  $a_{11} \times \text{제}(1,1) \text{ 소행렬식} - a_{12} \times \text{제}(1,2) \text{ 소행렬식}$   
 $+ a_{13} \times \text{제}(1,3) \text{ 소행렬식}$

$$\begin{bmatrix} 8 & 5 & 3 \\ 4 & 1 & 6 \\ 7 & 10 & 9 \end{bmatrix} \quad \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix}$$

$$\rightarrow \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix} - \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix}$$

$$+ \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix} \rightarrow \{8 \times (-51)\} - \{5 \times (-6)\} + \{3 \times (33)\} = -408 + 30 + 99 = -279$$

## 3) 3차 정사각행렬 행렬식 구하기

(3) 주어진 3차 정사각행렬 행렬식을 구해 보기

$$A = \begin{bmatrix} 8 & 5 & 3 \\ 4 & 1 & 6 \\ 7 & 10 & 9 \end{bmatrix}$$

```
import numpy as np  
A = np.array([[8,5,3],[4,1,6],[7,10,9]])  
np.linalg.det(A)
```

→ -278.99999999999994

## 2.NumPy 활용하기

### 1. 우리 반 학생들의 중간고사 등수 구하기

#### 1) 80, 75, 100, 90, 60인 점수의 등수 구하기

##### (1) 등수를 구하기 위한 절차

- 80, 75, 100, 90, 60 값을 갖는 배열을 생성함

[사용 함수]

`numpy.array(리스트)`

```
import numpy as np
score = np.array([80,75, 100, 90, 60])
```

- 생성한 배열의 요소 순위를 구함

[사용 함수]

오름차순 ➡ `array객체.argsort()`  
내림차순 ➡ `array객체.argsort()[::-1]`

```
desc_index = score.argsort( )[::-1]
```

## 1) 80, 75, 100, 90, 60인 점수의 등수 구하기

### (1) 등수를 구하기 위한 절차

- 추출한 인덱스에 등수 매기기

#### [사용 함수]

- 연속된 숫자 생성  
numpy.arange([시작값,] 끝값, [간격])  
※ 끝값은 포함하지 않음
- 배열 또는 리스트의 길이를 구하는 함수  
len(배열 또는 리스트)
- 특정 배열과 동일한 크기로 빈 배열 생성
  - numpy.empty\_like(배열): 초기화하지 않음
  - numpy.zeros\_like(배열): 초기화를 함
  - numpy.ones\_like(배열): 초기화를 함

```
# 등수를 저장하기 위한 빈 배열을 생성함
# rank=np.zeros_like(score)
# rank=np.ones_like(score)
rank=np.empty_like(score)
desc_index = score.argsort()[::-1]
# 내림차순으로 추출한 배열 인덱스에 등수를 저장함
rank[desc_index]=np.arange(1,len(score)+1)
```

## 1) 80, 75, 100, 90, 60인 점수의 등수 구하기

(2) 등수를 구하기 위한 코드를 Colab에서 실행해 보기

```
import numpy as np
score = np.array([80, 75, 100, 90, 60])
desc_index = score.argsort()[::-1]
rank = np.empty_like(score)
rank[desc_index] = np.arange(1, len(score)+1)
print(score)
print(rank)
```

```
[ 80  75 100  90  60]
[3  4  1  2  5]
```



## 2. 연립 방정식의 해 구하기

### 1) 2개의 미지수를 갖는 두 연립방정식의 해를 구하기

$$4x_1 - 2x_2 = 6 \quad 2x_1 + 3x_2 = 7$$

#### (1) 두 선형 연립방정식을 행렬로 변환

$$\begin{bmatrix} 4 & -2 \\ 2 & 3 \end{bmatrix} \cdot \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 6 \\ 7 \end{bmatrix}$$

#### (2) x, y 값 구하기

- np.linalg.solve를 이용하여 구하기

$$\begin{bmatrix} 4 & -2 \\ 2 & 3 \end{bmatrix} \cdot \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 6 \\ 7 \end{bmatrix}$$

[사용 함수]

연립방정식의 해: np.linalg.solve(행렬1, 행렬2)

```
import numpy as np
#계수 행렬
A = np.array([[4,-2],[2,3]])
#결과 행렬
R = np.array([6,7])
#방정식의 해 구하기
print(np.linalg.solve(A,R))
```

```
import numpy as np
#계수 행렬
A = np.array([[4,-2],[2,3]])
#결과 행렬
R = np.array([6,7])
#방정식의 해 구하기
print(np.linalg.solve(A,R))

[2. 1.]
```

## 1) 2개의 미지수를 갖는 두 연립방정식의 해를 구하기

### (2) x, y 값 구하기

- 역행렬 함수 np.linalg.inv를 이용하여 구하기

$$\begin{bmatrix} 4 & -2 \\ 2 & 3 \end{bmatrix} \cdot \begin{bmatrix} X1 \\ X2 \end{bmatrix} = \begin{bmatrix} 6 \\ 7 \end{bmatrix}$$

역행렬의 곱으로 변환

$$\begin{bmatrix} X1 \\ X2 \end{bmatrix} = \begin{bmatrix} 4 & -2 \\ 2 & 3 \end{bmatrix}^{-1} \cdot \begin{bmatrix} 6 \\ 7 \end{bmatrix}$$

[사용 함수]

역행렬을 구하는 함수: np.linalg.inv(행렬)

```
import numpy as np
#계수 행렬
A = np.array([[4,-2],[2,3]])
#결과 행렬
R = np.array([6,7])
#A행렬의 역행렬
A_inv = np.linalg.inv(A)
#방정식의 해 구하기
print(np.dot(A_inv,R))
```

```
import numpy as np
#계수 행렬
A = np.array([[4,-2],[2,3]])
#결과 행렬
R = np.array([6,7])
#A행렬의 역행렬
A_inv = np.linalg.inv(A)
#방정식의 해 구하기
print(np.dot(A_inv,R))

[2. 1.]
```

## 1) 2개의 미지수를 갖는 두 연립방정식의 해를 구하기

(3) 구한 해 x1, x2가 올바른지 확인하기

$$\begin{bmatrix} 4 & -2 \\ 2 & 3 \end{bmatrix} \cdot \begin{bmatrix} 2 \\ 1 \end{bmatrix} = \begin{bmatrix} 6 \\ 7 \end{bmatrix}$$

- array\_equal 함수 사용하기

[사용 함수]

두 행렬 비교: np.array\_equal(행렬1, 행렬2)

```
import numpy as np
#계수 행렬
A = np.array([[4,-2],[2,3]])
#결과 행렬
R = np.array([6,7])
#방정식의 해 구하기
X=np.linalg.solve(A,R)
#방정식의 해가 올바른지 확인하기
print(np.array_equal(np.dot(A,X),R))
```

## 3. 행렬의 곱 구하기

### 1) 주어진 행렬 A, B의 AB값 구하기

$$\begin{bmatrix} 8 & 5 & 3 \\ 4 & 1 & 6 \\ 7 & 10 & 9 \end{bmatrix} \quad \begin{bmatrix} 0 & 3 \\ 1 & 4 \\ 2 & 5 \end{bmatrix}$$

- (1) 행렬의 곱 : 교환 법칙( $AB=BA$ )이 성립
- (2) 행렬 곱셈의 교환 법칙은 동일한 shape를 가진 행렬에 한해서 가능함

## 1) 주어진 행렬 A, B의 AB값 구하기

### (3) \* 연산자를 이용하여 AB를 수행해 보기

```
import numpy as np
# 3 x 3 행렬을 생성하기
A = np.array([[8,5,3],[4,1,6],[7,10,9]])
# 2 x 2 행렬을 생성하기
B = np.array([[0,3],[1,4],[2,5]])
# * 연산자를 이용하여 행렬의 곱 AB를 구하기
print(A*B)
```

행렬 A, B의 shape(차원)가 서로 달라서 에러가 남

```
-----
ValueError                                Traceback (most recent call last)
<ipython-input-1-9db8d23be434> in <module>()
      5 B = np.array([[0,3],[1,4],[2,5]])
      6 # * 연산자를 이용하여 행렬의 곱 AB를 구하자.
----> 7 print(A*B)

ValueError: operands could not be broadcast together with shapes (3,3) (3,2)
```

## 1) 주어진 행렬 A, B의 AB값 구하기

(4) dot 함수를 이용하여 AB를 수행해 보기

```
import numpy as np
# 3 x 3 행렬을 생성하기
A = np.array([[8,5,3],[4,1,6],[7,10,9]])
# 2 x 2 행렬을 생성하기
B = np.array([[0,3],[1,4],[2,5]])
# dot 연산자를 이용하여 행렬의 곱 AB를 구하기
print(A.dot(B))
#print(np.dot(A,B))
```

```
▶ import numpy as np
# 3x3 행렬을 생성하자
A = np.array([[8,5,3],[4,1,6],[7,10,9]])
# 2x2 행렬을 생성하자
B = np.array([[0,3],[1,4],[2,5]])
# dot 연산자를 이용하여 행렬의 곱 AB를 구하자.
print(A.dot(B)) #print(np.dot(A,B))
```

```
↳ [[ 11  59]
    [ 13  46]
    [ 28 106]]
```

## 1) 주어진 행렬 A, B의 AB값 구하기

(5) dot 함수를 이용하여 BA를 수행해 보기

```
import numpy as np
# 3 x 3 행렬을 생성하기
A = np.array([[8,5,3],[4,1,6],[7,10,9]])
# 2 x 2 행렬을 생성하기
B = np.array([[0,3],[1,4],[2,5]])
# dot 연산자를 이용하여 행렬의 곱 AB를 구하기
print(B.dot(A))
```

행렬의 요소 개수가 작은 행렬에  
요소 개수가 많은 행렬을 곱하게 되면 에러가 남

```
-----
ValueError                                Traceback (most recent call last)
<ipython-input-4-7c8880e27fd2> in <module>()
      5 B = np.array([[0,3],[1,4],[2,5]])
      6 # dot 연산자를 이용하여 행렬의 곱 AB를 구하자.
----> 7 print(B.dot(A))

ValueError: shapes (3,2) and (3,3) not aligned: 2 (dim 1) != 3 (dim 0)
```